
Cleaning and Transforming Data

Feature engineering and transparent preparation

Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Learning objectives and route through the class

By the end of the core class, you should be able to distinguish missing values from reporting limits, check variable types and units, create a derived variable, standardize a numeric variable, compare discretization rules, and explain a cleaning decision while preserving its source record.

The class builds on Class A's missing-data mechanisms and on Week 1's summaries. We first review preparation choices, then use small examples to check their arithmetic, and finish with a monitoring-record workflow for Lab 02. Work in R or Python; the paired examples show equivalent operations in both languages. The optional extensions after the lab launch introduce model-based sampling and principal component analysis (PCA). They are not required for Lab 02.

1 Recap of missing data

1.1 A mechanism describes the recording process

A missing-data mechanism concerns the probability that a value is unavailable. It cannot be identified by looking at a single blank cell. The process that collected, measured, and recorded the observation matters.

Table 1. What missingness can depend on under each mechanism.

Mechanism	What missingness can depend on
MCAR	Neither observed nor unobserved values.
MAR	Observed values; after conditioning on them, not the missing values.
MNAR	Unobserved values still matter after conditioning on observed information.

MCAR is a sufficient condition for complete cases to form a random subset of otherwise eligible records. Complete-case summaries then avoid selection bias from missingness, although precision decreases. MAR permits missingness to vary with observed information. An appropriate analysis can account for that information, but MAR alone does not make an arbitrary imputation or deletion rule valid. MNAR means that dependence on unobserved values remains after conditioning on observed information. Observed variables may still explain part of that process. Its remaining assumptions need external information, an explicit model, or a sensitivity analysis [5, 9].

Do not infer that complete-case analysis must bias every estimate whenever MCAR fails. Some correctly specified conditional models remain valid under broader conditions. The relevant question is which quantity is being estimated and which missingness assumptions support that analysis.

1.2 Alternative strategies and their assumptions

Listwise deletion restricts an analysis to records complete on its required variables. Pairwise deletion uses the available records separately for each statistic. For example, a mean for one variable can use its observed records even when another variable is missing in those rows. Report the denominator for each summary so that different available subsets are visible. Different denominators answer different questions about the available records.

Mean, median, and mode replacement are simple forms of single imputation. Replacing missing values with the observed mean leaves that observed mean unchanged. This arithmetic identity does not show that it equals the population mean. For a fixed observed sample, adding copies of its mean reduces the sample variance by adding zero deviations to the numerator while increasing the divisor. Median or mode replacement need not preserve the mean. None of these rules restores the uncertainty of the unavailable measurements.

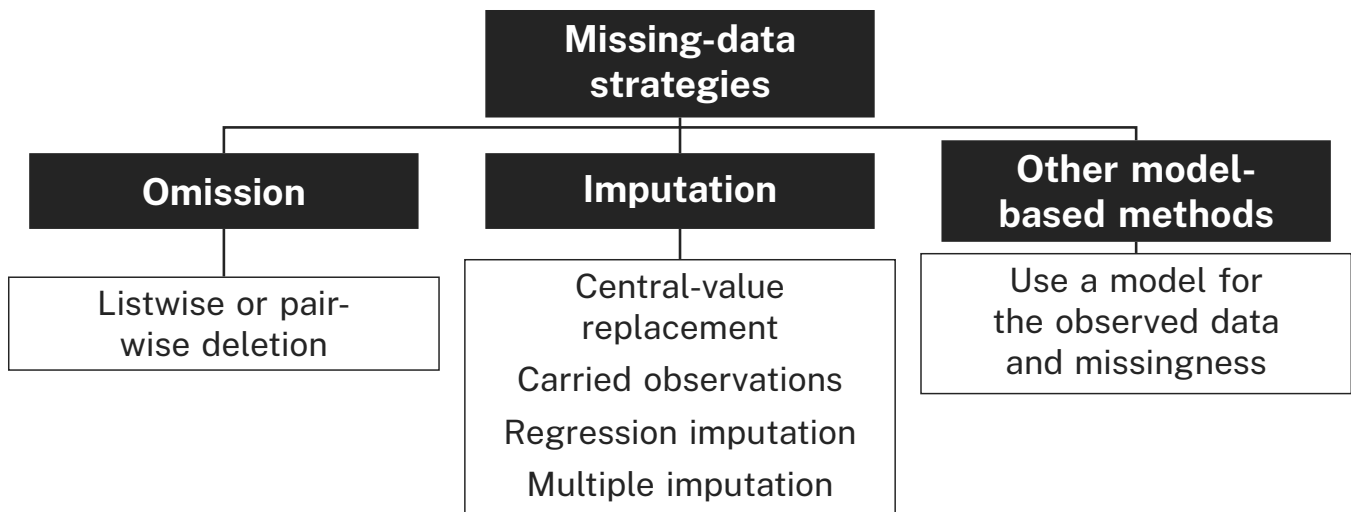


Figure 1. Alternative missing-data strategies. Branches identify alternatives, not a required sequence of operations.

Last observation carried forward (LOCF) repeats a previous value in a series. Baseline observation carried forward (BOCF) instead repeats a designated baseline value. Backward fill uses the next observed value and is not BOCF. Sort observations by time and keep separate units, people, or monitoring sites separate before any carry operation. A fill requires a defensible assumption about what happens between measurements; a leading missing value need not have a usable earlier observation.

Regression imputation predicts missing values from other variables. A fitted mean prediction alone omits residual variation. Multiple imputation creates several plausible completed datasets, analyses each, and pools the estimates and uncertainty. It requires compatible imputation and analysis assumptions; it is not simply repeating a deterministic fill. The `mice` package implements such workflows [1]. These model-based methods were introduced conceptually in Class A and are not required for Lab 02.

1.3 A small, explicit replacement example

Suppose an illustrative column contains 2, a missing value, 8, and 12. Its observed median is 8. The following example records which value was unavailable, keeps the source table, and makes the replacement in a separate copy.

R

```

raw <- data.frame(value = c(2, NA, 8, 12))
observed <- raw[!is.na(raw$value), , drop = FALSE]
cleaned <- raw
centre <- median(observed$value)
cleaned$was_missing <- is.na(raw$value)
cleaned$value[cleaned$was_missing] <- centre
cleaned
# An illustrative choice, not a recovered
  ↪ measurement.
  
```

Python

```

import pandas as pd

raw = pd.DataFrame({"value": [2, None, 8, 12]})
observed = raw.dropna(subset=["value"])
cleaned = raw.copy()
centre = observed["value"].median()
cleaned["was_missing"] = raw["value"].isna()
cleaned["value"] = cleaned["value"].fillna(centre)
print(cleaned)
  
```

This demonstrates how to retain a decision's evidence. It does not justify median replacement for a particular monitoring dataset. In Lab 02, retaining a missing value and reporting its count may be the most defensible choice.

2 Preparation within the course workflow

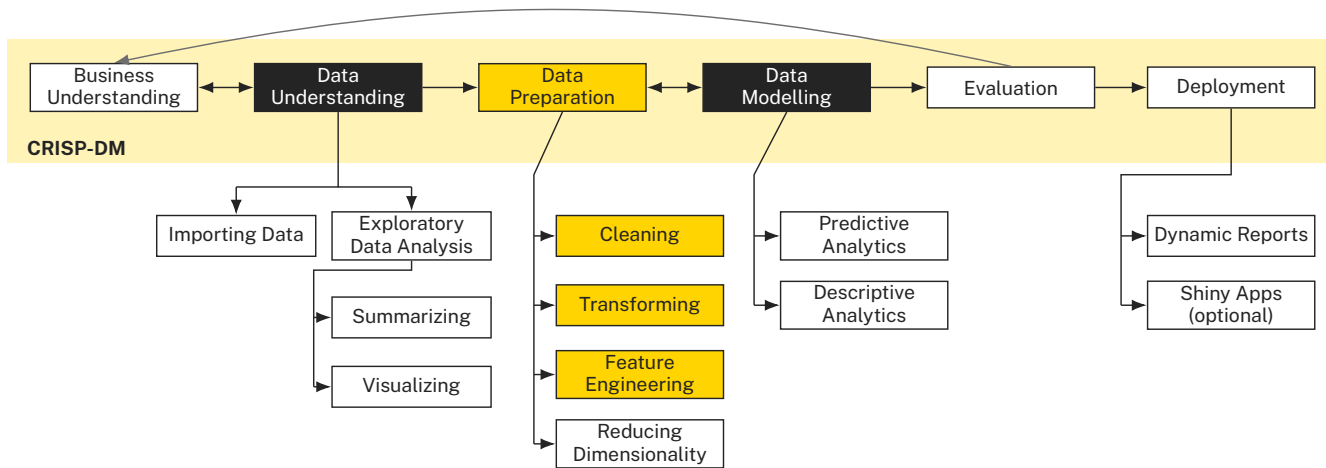


Figure 2. The course workflow with Data Preparation emphasized. Cleaning, transforming, feature engineering, and reduction are related preparation tasks.

Preparation is iterative. A summary may reveal an unexpected unit, a plot may show a recording error, and a revised interpretation may require a new import rule. There is no universal requirement to finish every preparation operation before inspecting the data again.

Table 2. Data problems and possible preparation responses.

Problem	Possible response
Variables have incompatible scales	Data transformation
A useful relationship needs a new column	Feature engineering
Records contain unavailable or invalid values	Data cleaning
Too many variables obscure an analysis	Dimensionality reduction

2.1 Feature engineering and units

A derived variable combines source measurements into a quantity useful for a question. Body mass index illustrates why units belong in the calculation. For a mass of 82 kg and height of 178 cm,

$$\text{BMI} = \frac{\text{mass in kg}}{(\text{height in m})^2} = \frac{82}{1.78^2} = 25.88 \text{ kg/m}^2.$$

The height is converted to metres before squaring. The example concerns arithmetic and documentation, not clinical interpretation.

R

```

measurements <- data.frame(
  mass_kg = c(82, 61, 95),
  height_cm = c(178, 165, 181))
measurements$bmi <- measurements$mass_kg /
  (measurements$height_cm / 100)^2
round(measurements$bmi, 2)
# 25.88 22.41 29.00

```

Python

```

import pandas as pd

measurements = pd.DataFrame({
  "mass_kg": [82, 61, 95],
  "height_cm": [178, 165, 181]})
measurements["bmi"] = measurements["mass_kg"] / (
  measurements["height_cm"] / 100)**2
print(measurements["bmi"].round(2).to_list())
# [25.88, 22.41, 29.0]

```

Both implementations keep the original measurements beside the derived value. The columns come from the same table, so corresponding rows describe the same person. R combines vectors by position and can recycle a shorter vector. pandas combines Series by their index labels. When combining values from different tables, first establish the correct row relationship rather than assuming equal lengths imply corresponding observations.

3 What it means to reduce a dataset

Reducing the number of features, reducing the number of observations, and reducing the number of distinct values are different choices. Fewer features can simplify computation or support a two-dimensional display. Fewer rows can make an initial inspection manageable, but may omit rare conditions. Binning reduces distinct numeric values to categories and loses within-bin detail.

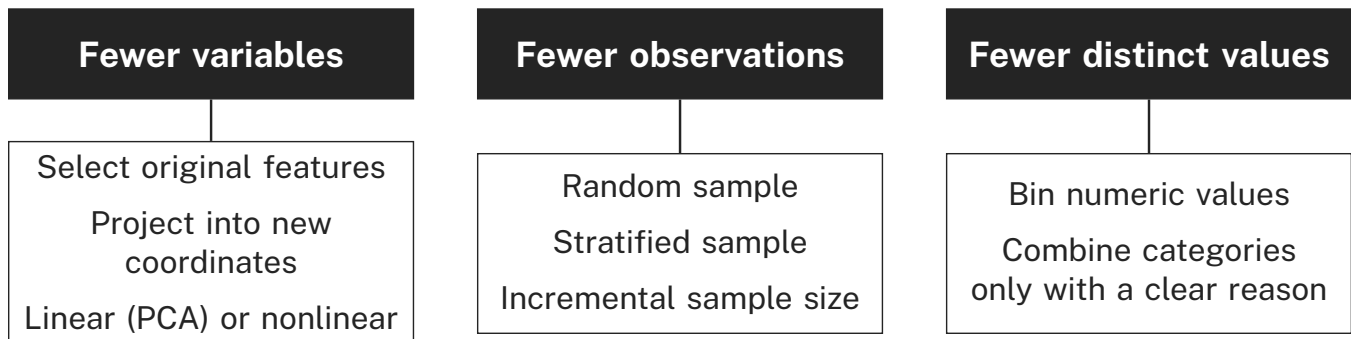


Figure 3. Three separate reduction choices. PCA is developed in the optional extension.

Feature selection retains some original variables. Feature projection creates new coordinates from combinations of variables. Linear and nonlinear projections are families of methods; PCA is a linear example. Their presence on the map does not require their use on this week's data.

3.1 Random and stratified sampling

A random sample chooses records through a defined chance mechanism. A stratified sample first separates records into groups, then samples within each group. Stratification can help retain rare groups; the group variable and allocation must follow the question. It cannot repair a biased source dataset.

The example has ten records, five from each of two sites. Both samples contain six records. The stratified sample takes three from each site, so its site proportions match those of this source table.

R

```
data <- data.frame(row_id = 1:10,
  site = rep(c("A", "B"), each = 5))
set.seed(3141)
idx <- sample(seq_len(nrow(data)), size = 6)
random_sample <- data[idx, ]
# Draw three rows from each site.
idx_by_site <- split(
  seq_len(nrow(data)), data$site)
chosen <- unlist(
  lapply(idx_by_site, sample, size = 3))
stratified_sample <- data[chosen, ]
table(stratified_sample$site)
# A 3, B 3
```

Python

```
import pandas as pd

data = pd.DataFrame({
  "row_id": range(1, 11),
  "site": ["A"]*5 + ["B"]*5})
random_sample = data.sample(
  n=6, random_state=3141)
# Draw three rows from each site.
stratified_sample = data.groupby("site").sample(
  n=3, random_state=3141)
print(stratified_sample["site"].value_counts())
# A 3, B 3
```

A seed makes a draw reproducible with the same software and algorithm. Equal seed values do not imply identical R and Python samples. An explicit integer sample size also avoids rounding differences. For example, 70 percent of 101 rows produces 70 rows after R's integer truncation, while pandas `sample(frac=0.7)` rounds to 71. Sampling a fraction separately within groups can change the overall count again.

Proportional allocation gives equal inclusion probabilities when the sampling fractions are equal. If groups are sampled at different rates, a population estimate must account for those probabilities. Sampling related station or time records also needs attention to dependence; random rows should not automatically be treated as independent observations. Lab 02 does not require subsampling or a training/test split.

4 Transforming numeric variables

4.1 Standardization

A variable's units affect its numerical spread. Some later methods compare variables through distances or variance, so one large scale can dominate their calculations. Centering and standardizing offer a particular way to change that comparison. For observed values with nonzero sample standard deviation,

$$z_i = \frac{x_i - \bar{x}}{s_x}.$$

Here $\bar{x}$ is the observed mean and s_x is the sample standard deviation using divisor $n - 1$. The resulting values have no units, observed mean zero, and sample standard deviation one.

For the Week 1 vector $x = 2, 4, 4, 5, 7, 9, 12, 41$, the mean is 10.5 and sample standard deviation is 12.72792. Selected results appear below.

Table 3. Selected standardized values, using the sample standard deviation.

x_i	$x_i - \bar{x}$	z_i
2	-8.5	-0.668
4	-6.5	-0.511
7	-3.5	-0.275
12	1.5	0.118
41	30.5	2.396

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
z <- scale(x)
attr(z, "scaled:center") # 10.5
attr(z, "scaled:scale")  # 12.72792
round(z[1], 4)           # -0.6678
scale(c(x, NA))[1]       # -0.6678231
# Only the last output is NA; observed values
#   remain.
```

Python

```
import numpy as np
from sklearn.preprocessing import StandardScaler

x = np.array([2, 4, 4, 5, 7, 9, 12, 41])
s = StandardScaler().fit(x.reshape(-1, 1))
print(s.mean_[0])           # 10.5
print(round(s.scale_[0], 4)) # 11.9059
print(round(s.transform(x[:, None])[0, 0], 4))
# -0.7139; use ddof=1 for the R convention:
print(round(((x - x.mean()) / x.std(ddof=1))[0], 4))
# -0.6678
```

R's `scale()` uses $n - 1$ when centering and scaling by the sample standard deviation. `StandardScaler` uses divisor n . For this vector the latter standard deviation is 11.9059, giving a first score of -0.7139 instead of -0.6678. Both implementations disregard missing values when estimating the center and spread and retain missing positions in the transformed output. All-missing or constant columns need

an explicit decision; division by zero is not a meaningful standardization. StandardScaler leaves a constant feature with a scaling factor of one, which does not give it variance one.

Standardization does not remove outliers, skewness, or dependence. The value 41 is still the extreme observation, now 2.396 sample standard deviations above the mean. A one-standard-deviation change is a numerical comparison, not proof that two variables have equal practical importance.

4.2 Discretization

Discretization replaces a numeric measurement with an interval label. This can support a categorical description, but it discards within-bin differences and makes the result depend on chosen boundaries. Use it when the question benefits from categories, not as automatic cleaning.

Equal-width boundaries are evenly spaced in measurement units. For four bins on the running vector, their spacing is $(41 - 2)/4 = 9.75$. Equal-frequency boundaries follow sample quantiles, here 2, 4, 6, 9.75, 41. Ties make the desired counts approximate.

Table 4. Equal-width bins with explicit boundaries.

Interval	Count	Members
[2, 11.75]	6	2,4,4,5,7,9
(11.75, 21.5]	1	12
(21.5, 31.25]	0	none
(31.25, 41]	1	41

Table 5. Equal-frequency bins based on sample quartiles.

Interval	Count	Members
[2, 4]	3	2, 4, 4
(4, 6]	1	5
(6, 9.75]	2	7, 9
(9.75, 41]	2	12, 41

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
edges <- seq(min(x), max(x), length.out = 5)
width_bins <- cut(x, breaks = edges,
                  include.lowest = TRUE)
quantiles <- quantile(x, probs = seq(0, 1, 0.25))
stopifnot(length(unique(quantiles)) == 5)
freq_bins <- cut(x, breaks = quantiles,
                include.lowest = TRUE)
table(width_bins) # 6 1 0 1
table(freq_bins) # 3 1 2 2
```

Python

```
import numpy as np
import pandas as pd

x = np.array([2, 4, 4, 5, 7, 9, 12, 41])
edges = np.linspace(x.min(), x.max(), 5)
width_bins = pd.cut(
    x, bins=edges, include_lowest=True)
freq_bins = pd.qcut(x, q=4)
print(pd.Series(width_bins).value_counts(
    sort=False))
print(pd.Series(freq_bins).value_counts(
    sort=False))
```

The first interval includes its lower boundary; each later interval is open on the left and closed on the right. Explicit edges make the intended calculation clearer than library-generated rounded labels. An integer argument to either library's cut can extend the range slightly to include endpoints. With explicit edges, the corresponding include.lowest or include_lowest option includes the minimum.

The counts are 6, 1, 0, 1 for equal width and 3, 1, 2, 2 for equal frequency. The last frequency bin contains both 12 and 41, hiding their 29-unit difference. Equal-width categorization also loses precise distances within each bin; it preserves equal boundary spacing, not the full original metric or shape.

A variable with many ties can produce repeated quantile boundaries. R rejects non-unique breaks, and `qcut` raises by default. A justified response is to use fewer distinct intervals, choose substantively meaningful boundaries, or retain the continuous variable. Report the resulting number of bins rather than promising four bins when the data cannot support them.

4.3 Representing categories

Nominal labels do not acquire an order simply because a computer needs a representation. Indicators encode membership directly.

R

```
site <- factor(c("A", "B", "A", "C"))
indicators <- model.matrix(~ site - 1)
indicators
# Three columns, one for each site; no intercept.
```

Python

```
import pandas as pd

site = pd.Series(["A", "B", "A", "C"])
indicators = pd.get_dummies(site, dtype=int)
print(indicators)
# Three columns, one for each site.
```

The example creates three indicators, one for each observed category. Neither implementation assigns a numerical order to the site names. Lab 02 requires clear variable types and documented labels; it does not require a particular model representation.

5 From monitoring records to a documented table

5.1 Separate the meaning of unavailable measurements

Consider an illustrative concentration column documented in mg/L. These five strings describe different situations.

Table 6. A small monitoring example with distinct value-status categories.

Raw field	Status	Interpretation
0	measured	Exact recorded zero in the stated unit.
blank	missing	No usable measurement was recorded.
<0.1	censored	Below 0.1 mg/L; retain 0.1 as a limit, not a value.
oops	invalid	Unexpected token; investigate the original record.
0.24	measured	Exact recorded value, subject to provider quality flags.

The string <0.1 reports an upper limit, not an exact measurement of 0.1 and not a measured zero. The value may be unavailable for exact arithmetic while its limit still carries information. A blank is a missing record under this example's documentation. The token oops is invalid under the numeric schema and needs investigation. Provider flags may add further restrictions, such as a provisional reading or a failed instrument check.

R

```
raw <- data.frame(
  value_text = c("0", "", "<0.1", "oops", "0.24"),
  unit = rep("mg/L", 5),
  stringsAsFactors = FALSE)
cleaned <- raw
cleaned$status <- c(
  "measured", "missing", "censored",
  "invalid", "measured")
cleaned$value_mg_L <- c(0, NA, NA, NA, 0.24)
cleaned$limit_mg_L <- c(NA, NA, 0.1, NA, NA)
stopifnot(identical(
  raw$value_text, cleaned$value_text))
# These statuses follow the stated rules.
# Original strings and units remain.
```

Python

```
import pandas as pd

raw = pd.DataFrame({
  "value_text":
    ["0", "", "<0.1", "oops", "0.24"],
  "unit": ["mg/L"]*5})
cleaned = raw.copy()
cleaned["status"] = [
  "measured", "missing", "censored",
  "invalid", "measured"]
cleaned["value_mg_L"] = [
  0, None, None, None, 0.24]
cleaned["limit_mg_L"] = [
  None, None, 0.1, None, None]
assert raw["value_text"].equals(
  cleaned["value_text"])
```

These assignments are deliberately explicit for five known records. A real file needs a documented rule derived from the provider's data dictionary. Read the original fields as strings, inspect all distinct tokens, and classify them before numeric conversion. Preserve literal tokens such as NA until the documentation establishes whether they mean unavailable, a category, or something else. Do not suppress conversion warnings without examining the records that caused them.

5.2 Apply the documented rule to every row

A reusable operation applies one declared rule to every matching record. For this small example only, a blank means missing, unsigned decimal strings are exact measurements, and a less-than sign followed by such a decimal is a reporting limit. The patterns below express those rules. `[0-9]+` means one or more digits, `([.][0-9]+)?` permits a decimal part, and the anchors or full-match operation require the whole string to match. Unrecognized tokens remain flagged as invalid rather than being silently reclassified as missing.

R

```
raw <- data.frame(value_text = c("0", "", "<0.1", "
  ↪ oops", "0.24"),
  unit = rep("mg/L", 5))
cleaned <- raw
text <- trimws(raw$value_text)
missing <- text == "" # This provider defines blank
  ↪ as missing.
censored <- grepl("<[0-9]+([.][0-9]+)?$", text)
measured <- grepl("^[0-9]+([.][0-9]+)?$", text)
cleaned$status <- ifelse(missing, "missing", ifelse(
  ↪ censored,
  "censored", ifelse(measured, "measured", "invalid"
  ↪ )))
cleaned$value_mg_L <- NA_real_
cleaned$value_mg_L[measured] <- as.numeric(text[
  ↪ measured])
cleaned$limit_mg_L <- NA_real_
cleaned$limit_mg_L[censored] <- as.numeric(sub("<",
  ↪ "", text[censored]))
cleaned$value_uug_L <- 1000 * cleaned$value_mg_L
cleaned
```

Python

```
import pandas as pd
raw = pd.DataFrame({"value_text": ["0", "", "<0.1",
  ↪ "oops", "0.24"],
  "unit": ["mg/L"]*5})
cleaned = raw.copy()
text = raw["value_text"].str.strip()
missing = text.eq("") # This provider defines blank
  ↪ as missing.
censored = text.str.fullmatch(r"<[0-9]+([.][0-9]+)?"
  ↪ )
measured = text.str.fullmatch(r"[0-9]+([.][0-9]+)?")
cleaned["status"] = "invalid"
for mask, label in [(missing, "missing"), (censored,
  ↪ "censored"), (measured, "measured")]:
  cleaned.loc[mask, "status"] = label
cleaned["value_mg_L"] = pd.to_numeric(text.where(
  ↪ measured))
cleaned["limit_mg_L"] = pd.to_numeric(text.where(
  ↪ censored).str.removeprefix("<"))
cleaned["value_uug_L"] = 1000 * cleaned["value_mg_L"]
print(cleaned)
```

The code converts only validated measurement strings and stores limit values separately. It also demonstrates the explicit mg/L-to-micrograms/L conversion. These are deliberately narrow example rules; real records may use signs, scientific notation, other missing codes, or different reporting-limit

formats. Inspect those conventions before adapting the rules. The provider documentation and token audit determine the classification; the code makes its application reproducible across all rows.

The cleaned copy retains the original text and units, a status field, a column for exact usable measurements, and a separate column for the reporting limit. It does not replace a censored result with zero or half the limit. Such a choice would require an additional assumption and sensitivity analysis, which this example does not make.

5.3 Check units and preserve decision evidence

Before combining observations, identify station, time, parameter, unit, and quality-flag fields. Some tables contain one row per station-time-parameter reading, rather than one row per station. State that observational unit so that counts and joins retain their meaning.

Compatible unit conversions need a named rule. For example, $1 \text{ mg/L} = 1000 \mu\text{g/L}$. Retain the original unit field and place a conversion in a new clearly named column. Do not combine concentrations from different parameters merely because they share a unit. Nor should a numerically unusual result be removed without evidence of an error.

Document the number of exact, missing, censored, invalid, and excluded records. A mean computed only from exact usable measurements describes that subset; when censoring or exclusions are substantial it may not represent all sampled conditions. State the denominator and limitation beside the summary.

5.4 Integrated practice and Lab 02

1. Read one approved monitoring record and identify its observational unit.
2. Inspect types, units, missing tokens, reporting limits, and quality flags.
3. Create a cleaned copy and record a reason for every changed or excluded value.
4. Compare summaries before and after preparation while retaining the raw record.
5. If categories help the question, compare two binning rules on one suitable numeric variable.

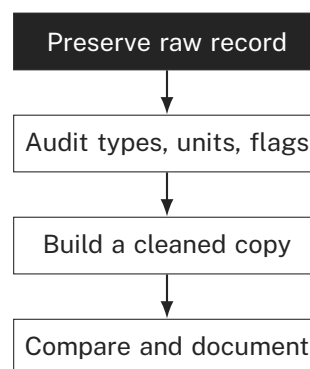


Figure 4. A preparation workflow that keeps raw evidence separate from the analytical copy.

The small worked examples are ungraded practice. The Lab 02 handout defines the assessed dataset options, deliverables, rubric, and submission package. No PCA, model fitting, or training/test split is required. Use one approved language. Follow the syllabus for the private submission and later portfolio release, and preserve the same submitted commit and its recorded history.

Integrated lab | Lab 02 | Cleaning water and air quality data

Build a transparent analytical table from one official monitoring record.

- **Carry forward.** Week 01 summaries, units, missingness, reporting limits, and unusual-value investigation.
- **Start in class.** Choose one verified dataset, work through the selection checklist, and set up the R or Python analysis.
- **Assessment source.** week-02/Assessment/lab-02

6 Next class

The next class returns to Data Understanding through visualization. Plots help check whether summaries and cleaning decisions faithfully represent the observations. This is a return to an earlier workflow stage, not evidence that the stages must always be completed in one fixed order.

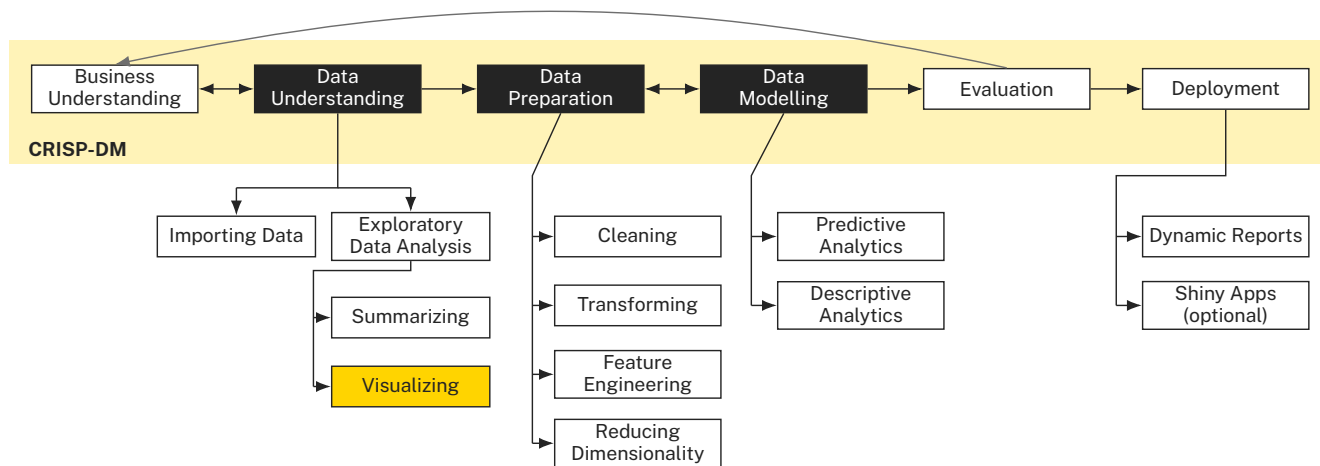


Figure 5. The same course map with visualization highlighted for the next class.

7 Optional extension on model-based sampling

These optional examples introduce the model-based sampling and projection ideas that later classes develop. They are not required for Lab 02. Model fitting, prediction, and performance estimation are developed in later classes.

7.1 Training, validation, and test roles

Training observations fit a model. Validation observations help choose among candidates or settings. Test observations support a final evaluation after those choices are fixed. A data-dependent transform, such as a fitted scaler, uses the training observations and is then applied unchanged to the other roles. Fitting on all observations before a model comparison transfers information from the evaluation rows into the training process.

This restriction concerns model evaluation. A descriptive summary of the current dataset does not need an arbitrary training/test split. Related rows from the same station, person, or time period may also require a grouped or ordered evaluation design; a random row split does not guarantee independence.

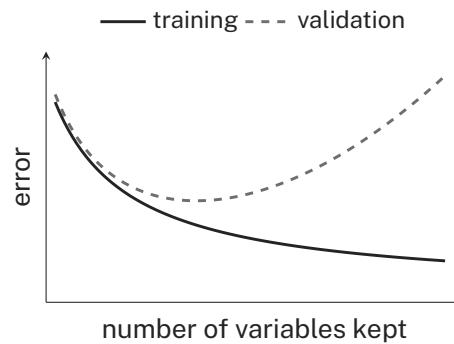


Figure 6. A schematic comparison of training and validation error as more variables are kept. The curves are illustrative and do not determine an optimal feature count.

Overfitting occurs when a model follows training-specific variation and loses accuracy on new observations. Reducing features can simplify a model and reduce computation, but it can also discard useful information. A validation comparison helps assess that tradeoff. Test observations should not repeatedly select the number of features.

7.2 Optional context on model encodings

A linear model's intercept represents its baseline prediction when the numeric inputs are zero. If every category has an indicator, the indicators sum to one; including all of them alongside an intercept introduces redundancy. One reference indicator is commonly omitted in that model. R's default treatment contrasts and an encoder using `drop="first"` can implement this choice. It is not a universal rule for all R or Python methods. New categories need an explicit policy when predicting with a fitted model; encoding an unknown label as all zeros does not show that it was observed during fitting.

7.3 Incremental sampling

Incremental sampling begins with a smaller training sample and increases its size. Compare candidates on the same validation set using a planned tolerance and a repeatability check. A small change should be assessed against variation between repetitions, and relevant groups still need adequate representation. One flat increment is not proof that a larger dataset cannot help.

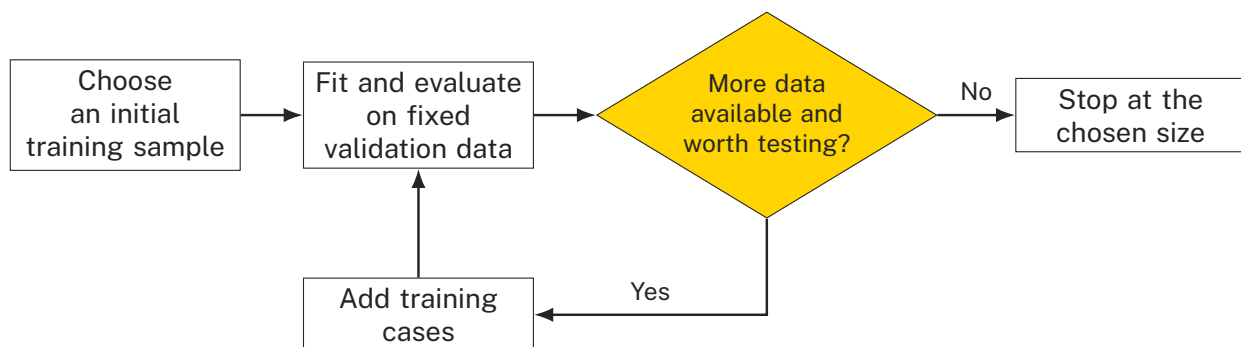


Figure 7. An incremental sampling loop with a continuation decision and a stopping condition.

7.4 Several samples and several fitted models

A method can be fitted separately to several samples. Each fitted model then predicts the same new case. Numeric predictions can be averaged and class predictions can be combined by voting. Bagging uses bootstrap resamples of a training set; those samples need not have fewer rows than the original dataset.

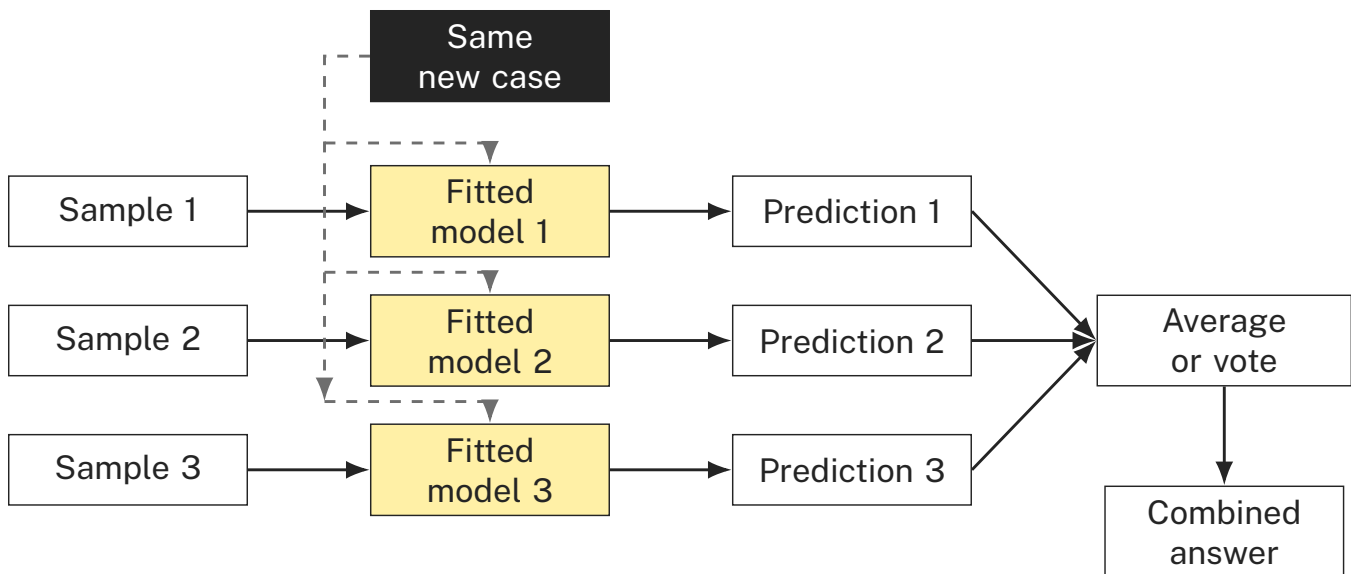


Figure 8. Separate fitted models receive the same new case; their predictions are aggregated.

8 Optional extension on principal component analysis

8.1 Directions of variation

PCA constructs new numeric coordinates that are linear combinations of the input variables [4, 6]. It orders those directions by the variance they carry. If a two-variable cloud is long and narrow, a coordinate along its long direction may approximate much of its variation. Here “information” means variance; a low-variance direction can still be important for a particular scientific question or prediction target.

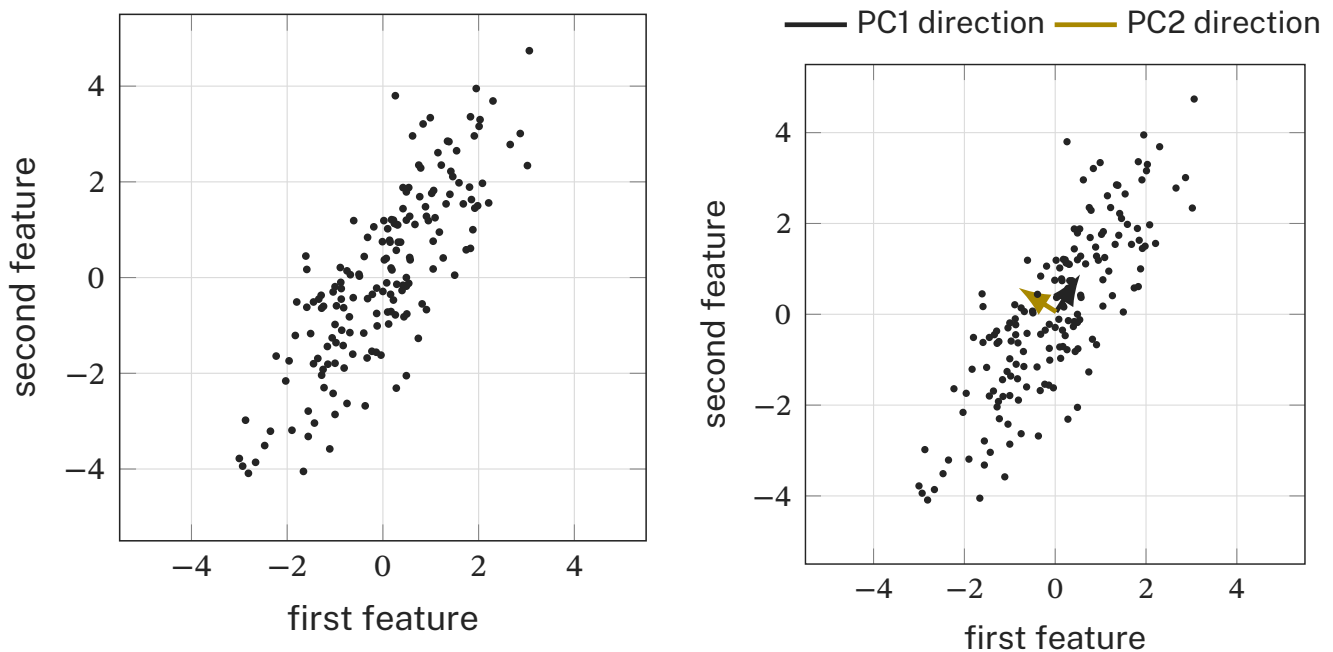


Figure 9. The same illustrative cloud before and after adding PC1 and PC2 directions. Arrows have unit length and do not encode variance. Both axes use the same physical unit scale.

The first direction maximizes projected variance subject to unit length. The next direction is perpendicular to the first and maximizes the remaining variance. Feature projection differs from selecting

a subset of original columns; computing a projected coordinate generally still requires all measurements with nonzero weights in that coordinate.

8.2 Loadings and standardized iris measurements

The worked example uses the four numeric measurements of the iris dataset. Species labels are excluded from fitting. We standardize each measurement so that the example compares its spread with the others on the same scale. Scaling is a choice, not a requirement of all PCA. A covariance PCA can be appropriate when the original measurement scales should be retained. Missing values and constant columns need an explicit treatment before this standardized fit.

R

```
features <- iris[, 1:4]
p <- prcomp(features, scale. = TRUE)
round(p$rotation, 4)      # one component per
  ↪ column
round(p$x[1, 1], 6)      # -2.257141
round(p$sdev^2 / sum(p$sdev^2), 4)
# 0.7296 0.2285 0.0367 0.0052
```

Python

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

features = load_iris(as_frame=True).data
Z = StandardScaler().fit_transform(features)
p = PCA().fit(Z)
print(p.components_.round(4)) # one component per
  ↪ row
print(p.explained_variance_ratio_.round(4))
# [0.7296, 0.2285, 0.0367, 0.0052]
```

Table 7. Loadings from sample-standardized iris measurements. Component signs follow the R example and may be reversed by another implementation.

	PC1	PC2	PC3	PC4
Sepal.Length	0.5210659	−0.3774176	0.7195664	0.2612863
Sepal.Width	−0.2693474	−0.9232957	−0.2443818	−0.1235096
Petal.Length	0.5804131	−0.0244916	−0.1421264	−0.8014492
Petal.Width	0.5648565	−0.0669420	−0.6342727	0.5235971

Let z_{SL} , z_{SW} , z_{PL} , and z_{PW} denote the centered, sample-standardized sepal length, sepal width, petal length, and petal width. Reading down the first loading column,

$$\begin{aligned} \text{PC1} &= 0.5210659z_{SL} - 0.2693474z_{SW} \\ &\quad + 0.5804131z_{PL} + 0.5648565z_{PW}. \end{aligned}$$

The first iris observation has a score of -2.257141 on this component. Multiplying raw centimetre measurements by these weights would not reproduce the standardized analysis. PC1 contrasts sepal width with the other three measurements; PC2 is dominated by sepal width.

Each loading column is a unit-length direction and the columns are mutually orthogonal. R stores one component per column in rotation; scikit-learn stores one per row in components_. Component signs are arbitrary. Reversing a direction and all its scores describes the same projection with the axis orientation reversed.

R's sample standardization and StandardScaler's population standardization make the input matrices differ by the common factor $\sqrt{(n-1)/n}$ for this complete dataset. Up to component signs, R's scores equal Python's scores multiplied by that factor. Loadings and explained-variance proportions agree. prcomp uses singular value decomposition; no eigenvector derivation or manual decomposition is required for the example.

8.3 Scores and a two-dimensional view

Multiplying the standardized data by the loading matrix gives scores, one coordinate per observation and component. The score plot places each iris observation on its first two coordinates and identifies species after fitting.

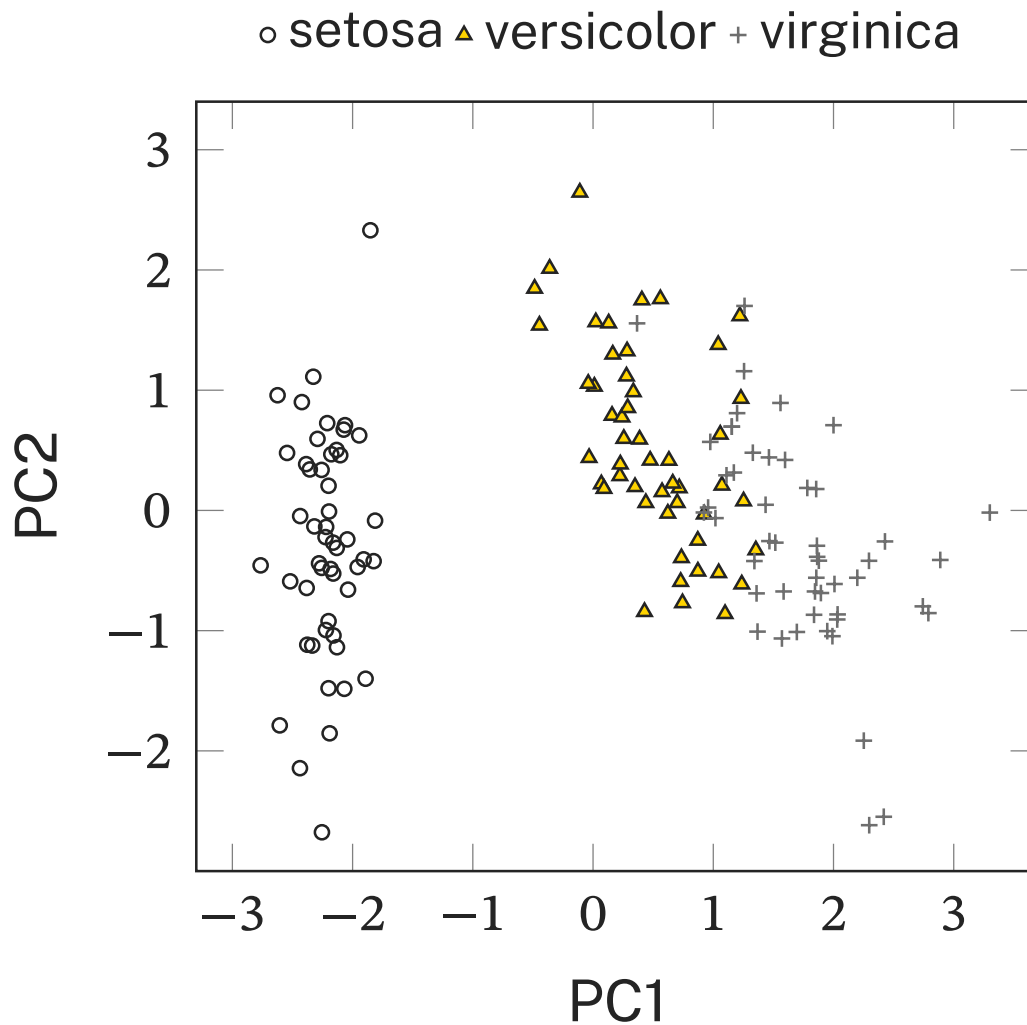


Figure 10. The first two sample-standardized iris scores. Species shapes are labels for interpretation, not inputs to PCA.

R

```
features <- iris[, 1:4]
p <- prcomp(features, scale. = TRUE)
png("iris-pca-R.png", width = 700, height = 700)
plot(p$x[, 1:2], pch = as.integer(iris$Species),
     xlab = "PC1", ylab = "PC2", asp = 1)
legend("topright", legend = levels(iris$Species),
     pch = 1:3, bty = "n")
dev.off()
```

Python

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
data = load_iris()
Z = StandardScaler().fit_transform(data.data)
scores = PCA().fit_transform(Z)
for k, marker in enumerate(["o", "^", "+"]):
    points = scores[data.target == k]
    plt.scatter(
        *points[:, :2].T, marker=marker,
        label=data.target_names[k])
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.axis("equal"); plt.legend()
plt.savefig("iris-pca-Python.png")
plt.close()
```

These executable examples save the plot explicitly. A reflected axis between R and Python is an allowed sign choice. The displayed grouping is a descriptive property of this dataset; it does not establish predictive accuracy on new observations.

8.4 Explained variance and retained components

The component proportions are 0.7296, 0.2285, 0.0367, and 0.0052. Their cumulative proportions are 0.7296, 0.9581, 0.9948, and 1.0000.

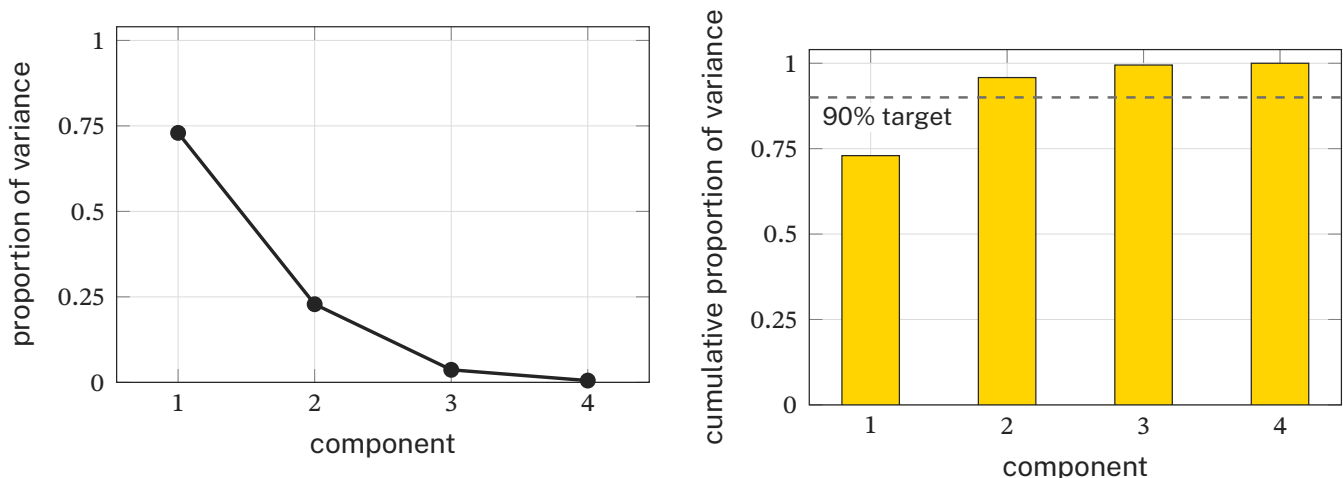


Figure 11. Per-component variance on the left and cumulative retained variance on the right. The dashed line marks an illustrative 90 percent target.

Two components retain 95.81 percent of the variance and therefore exceed a chosen 90 percent target. That target is neither a theorem nor a guarantee that discarded variation is irrelevant. Decide how much loss the intended use can tolerate. If the representation is selected to help a predictive model, make that choice within the training/validation process introduced above.

8.5 Biplots and their display scales

A biplot combines rescaled observation scores with arrows for the original variables [3]. The first two principal components provide the displayed projection. Points use the bottom and left scales; arrows use the top and right scales in the course figure.

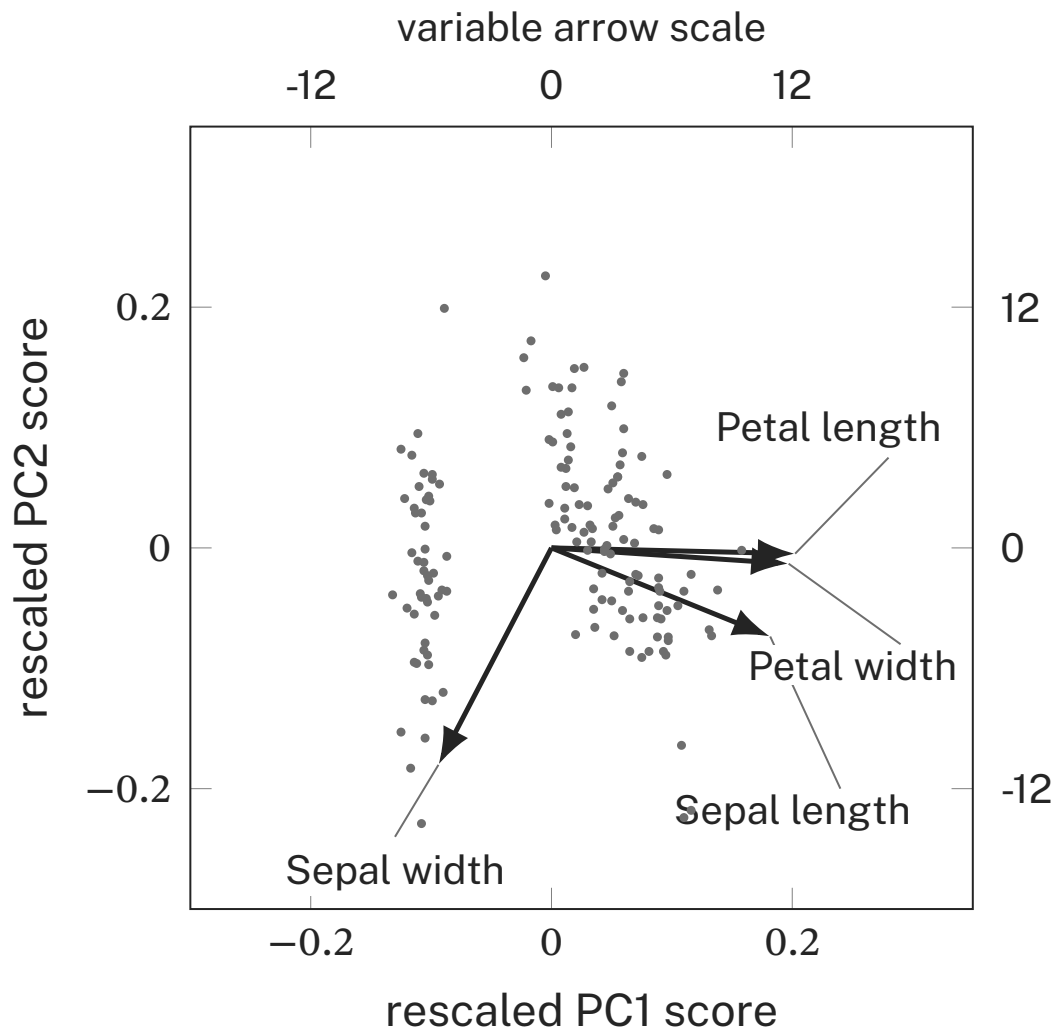


Figure 12. An iris biplot with separate scales for rescaled points and variable arrows. The same figure is used in the slide deck.

Arrows pointing in similar directions indicate variables that vary together within the retained projection. Their angular interpretation is approximate when the two components represent a variable poorly. Arrow length and point distance also depend on the chosen biplot scaling; neither should be read as a raw measurement.

R's default biplot uses $\lambda_j = s_j \sqrt{n}$, where s_j is the standard deviation of component j . It divides scores by λ_j and multiplies loadings by it before applying display limits. The following Python example uses sample-standardized inputs and the same rescaled quantities. Its arrow factor of 1/60 is an explicit display choice. It illustrates the quantities without claiming that Matplotlib reproduces all R drawing defaults.

R

```
p <- prcomp(iris[, 1:4], scale. = TRUE)
# R's default biplot rescaling.
lam <- p$sdev[1:2] * sqrt(nrow(iris))
points <- sweep(p$x[, 1:2], 2, lam, "/")
arrows <- sweep(p$rotation[, 1:2], 2, lam, "*")
png("iris-biplot-R.png", width = 700, height = 700)
biplot(p)
dev.off()
```

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

features = load_iris(as_frame=True).data
Z = (features - features.mean()) / (
    features.std(ddof=1))
p = PCA().fit(Z)
lam = np.sqrt(p.explained_variance_[2] * len(Z))
points = p.transform(Z)[:2] / lam
arrows = p.components_[2].T * lam
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(*points.T, s=5)
ax.set(xlim=(-0.26, 0.26), ylim=(-0.26, 0.26), xlabel="PC1", ylabel="PC2")
ax.set_aspect("equal")
for (u, v), name in zip(arrows / 60, features.columns):
    ax.annotate("", xy=(u, v), xytext=(0, 0), arrowprops={"arrowstyle": "->"})
    ax.text(u, v, name, fontsize=7)
# Arrow coordinates use a shared factor 1/60 for display.
fig.savefig("iris-biplot-Python.png")
plt.close(fig)
```

9 Check your interpretation

1. Why are a measured zero and a result below a reporting limit different?
2. Which fields let another reader recover the raw record after cleaning?
3. What does standardization change, and which aspects of a distribution remain?
4. Why do tied values prevent exactly equal-frequency bins in the running example?
5. In the optional PCA example, why must the displayed loadings multiply standardized values?
6. Why does an attractive optional PCA plot not by itself demonstrate predictive accuracy?

Optional further reading

These are the same follow-up links shown in the slides. They explain the named operations and are not additional assessed prerequisites.

- R: scale and Python: StandardScaler explain centering, divisors, and missing values.
- R: cut and Python: qcut document interval boundaries and repeated quantiles.
- R: prcomp and Python: PCA support the optional projection example.

The references below retain the original PCA and biplot papers and the missing-data sources for readers who want further detail.

References

- [1] S. van Buuren and K. Groothuis-Oudshoorn. “mice: multivariate imputation by chained equations in R”. In: *Journal of Statistical Software* 45.3 (2011), pp. 1–67.

- [2] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.
- [3] K. R. Gabriel. “The biplot graphic display of matrices with application to principal component analysis”. In: *Biometrika* 58.3 (1971), pp. 453–467.
- [4] H. Hotelling. “Analysis of a complex of statistical variables into principal components”. In: *Journal of Educational Psychology* 24 (1933), pp. 417–441, 498–520.
- [5] R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. 3rd ed. Wiley, 2019.
- [6] K. Pearson. “On lines and planes of closest fit to systems of points in space”. In: *Philosophical Magazine* 2.11 (1901), pp. 559–572.
- [7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. “Scikit-learn: machine learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [8] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.
- [9] D. B. Rubin. “Inference and missing data”. In: *Biometrika* 63.3 (1976), pp. 581–592.